

Control as Variational Inference

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

Control as inference recap

A probabilistic graphical model of decision making

~~$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)$$

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$~~

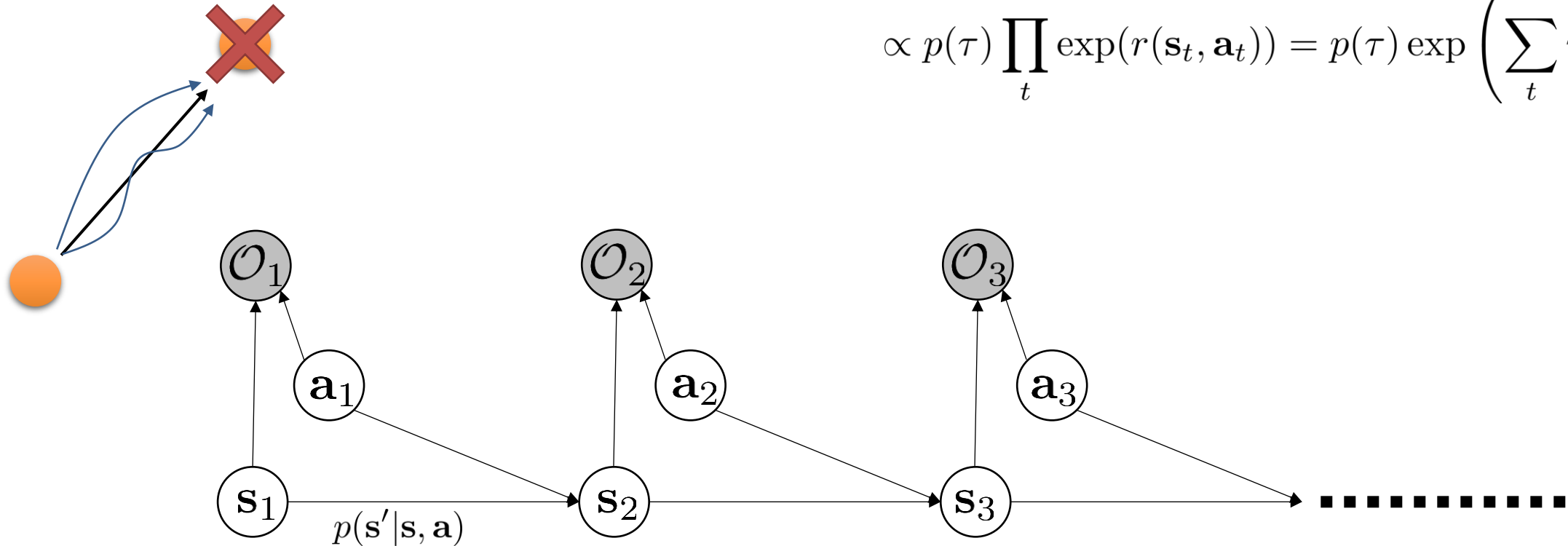
$$p(\underbrace{\mathbf{s}_{1:T}, \mathbf{a}_{1:T}}_{\tau}) = ?? \quad \text{no assumption of optimal behavior!}$$

$$p(\tau | \mathcal{O}_{1:T})$$

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$$

$$p(\tau | \mathcal{O}_{1:T}) = \frac{p(\tau, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})}$$

$$\propto p(\tau) \prod_t \exp(r(\mathbf{s}_t, \mathbf{a}_t)) = p(\tau) \exp\left(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right)$$



Policy computation

for $t = T - 1$ to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log E[\exp(V_{t+1}(\mathbf{s}_{t+1}))]$$

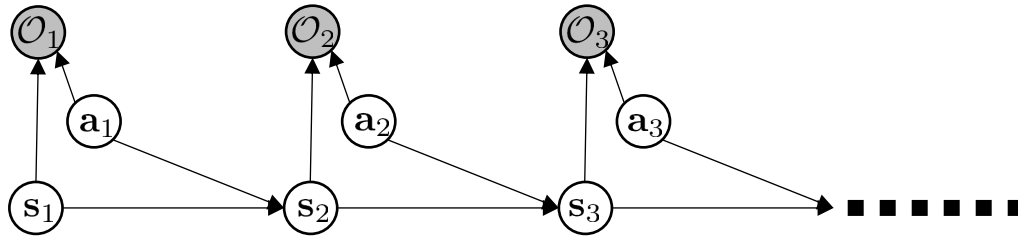
$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) \mathbf{a}_t$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} \quad \begin{array}{l} V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t) \\ Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t) \end{array}$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)) = \exp(A_t(\mathbf{s}_t, \mathbf{a}_t))$$

Summary

1. Probabilistic graphical model for optimal control



2. Control = inference (similar to HMM, EKF, etc.)

3. Very similar to dynamic programming, value iteration, etc. (but “soft”)

Part 2:

Control as variational inference

The optimism problem

for $t = T - 1$ to 1:

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})]$$

“optimistic” transition
(not a good idea!)

$$\beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)]$$

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \overbrace{\log E[\exp(V_{t+1}(\mathbf{s}_{t+1}))]}$$

let $V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t)$

let $Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t)$

why did this happen?

the inference problem: $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T} | \mathcal{O}_{1:T})$

marginalizing and conditioning, we get: $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T})$ (the policy)

“given that you obtained high reward, what was your action probability?”

marginalizing and conditioning, we get: $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t, \mathcal{O}_{1:T}) \neq p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$

“given that you obtained high reward, what was your transition probability?”

Addressing the optimism problem

marginalizing and conditioning, we get: $p(\mathbf{a}_t|\mathbf{s}_t, \mathcal{O}_{1:T})$ (the policy) $\longleftarrow$ we want this

“given that you obtained high reward, what was your action probability?”

marginalizing and conditioning, we get: $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \mathcal{O}_{1:T}) \neq p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ $\longleftarrow$ but not this!

“given that you obtained high reward, what was your transition probability?”

“given that you obtained high reward, what was your action probability,

given that your transition probability did not change?”

can we find another distribution $q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$ that is close to $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}|\mathcal{O}_{1:T})$ but has dynamics $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$

where have we seen this before?

let $\mathbf{x} = \mathcal{O}_{1:T}$ and $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$ find $q(\mathbf{z})$ to approximate $p(\mathbf{z}|\mathbf{x})$

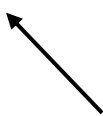
let's try variational inference!

Control via variational inference

let $q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) q(\mathbf{a}_t | \mathbf{s}_t)$

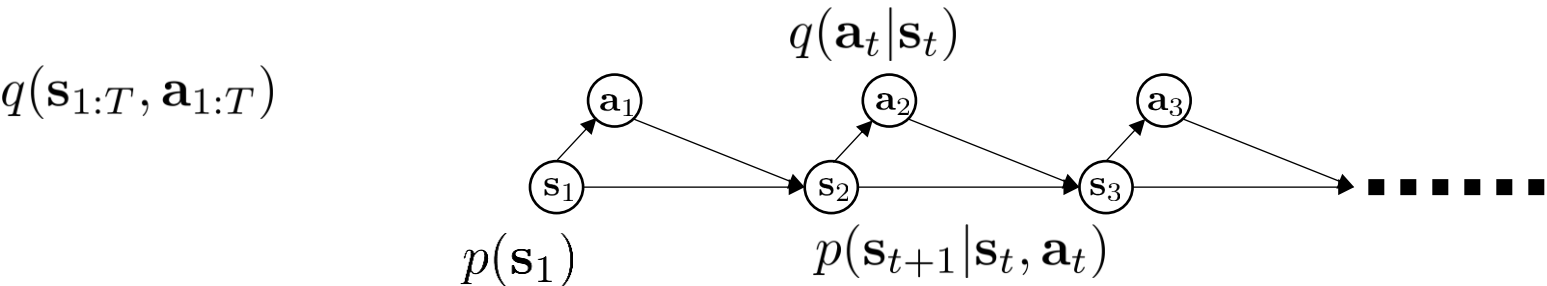
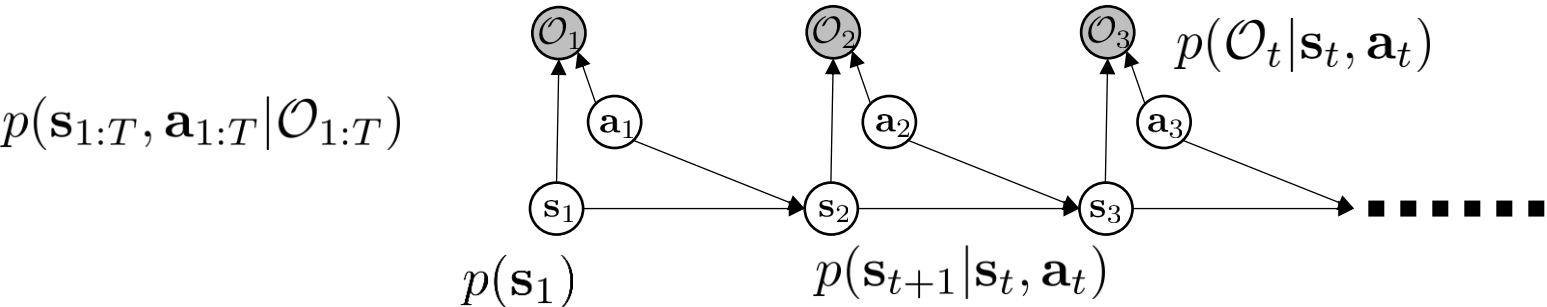


same dynamics and
initial state as p



only new thing

let $\mathbf{x} = \mathcal{O}_{1:T}$ and $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$

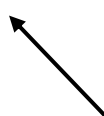


The variational lower bound

$$\log p(\mathbf{x}) \geq E_{\mathbf{z} \sim q(\mathbf{z})} [\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z})]$$

let $\mathbf{x} = \mathcal{O}_{1:T}$ and $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$

the entropy $\mathcal{H}(q)$



$$\text{let } q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) = \underbrace{p(\mathbf{s}_1)}_{\text{green}} \underbrace{\prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{\text{orange}} q(\mathbf{a}_t | \mathbf{s}_t)$$

$$\begin{aligned} \log p(\mathcal{O}_{1:T}) \geq E_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} [& \log p(\mathbf{s}_1) + \sum_{t=1}^T \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) + \sum_{t=1}^T \log p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) \\ & \underbrace{- \log p(\mathbf{s}_1)}_{\text{green}} - \underbrace{\sum_{t=1}^T \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{\text{orange}} - \sum_{t=1}^T \log q(\mathbf{a}_t | \mathbf{s}_t)] \end{aligned}$$

$$= E_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) - \log q(\mathbf{a}_t | \mathbf{s}_t) \right]$$

$$= \sum_t E_{(\mathbf{s}_t, \mathbf{a}_t) \sim q} [r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(q(\mathbf{a}_t | \mathbf{s}_t))]$$

maximize reward and maximize action entropy!

Backward pass summary - variational

for $t = T - 1$ to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + E[(V_{t+1}(\mathbf{s}_{t+1}))]$$

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t$$

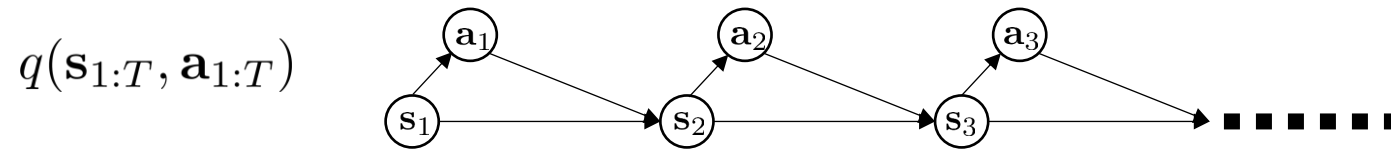
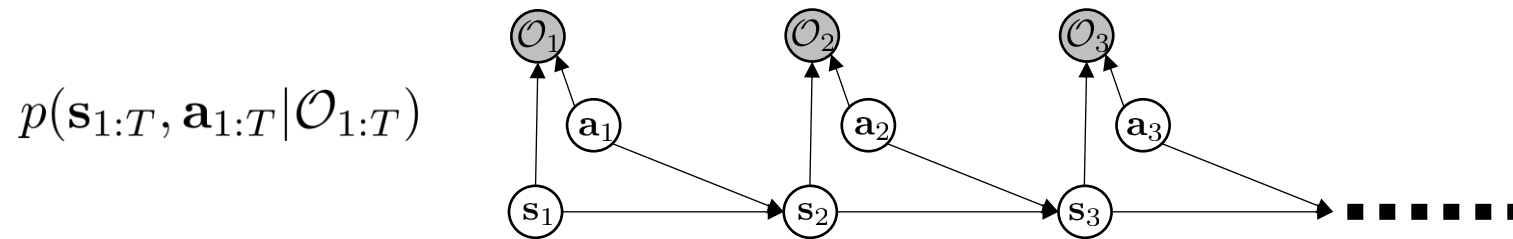
value iteration algorithm:

- 
1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')]]$
 2. set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

soft value iteration algorithm:

- 
1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')]]$
 2. set $V(\mathbf{s}) \leftarrow \text{soft max}_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

Summary



$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t \quad Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + E[(V_{t+1}(\mathbf{s}_{t+1}))]$$

variants:

discounted SOC: $Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma E[V_{t+1}(\mathbf{s}_{t+1})]$

explicit temperature: $V_t(\mathbf{s}_t) = \alpha \log \int \exp\left(\frac{1}{\alpha} Q_t(\mathbf{s}_t, \mathbf{a}_t)\right) d\mathbf{a}_t$

Part 3:

Maximum entropy RL algorithms

Q-learning with soft optimality

standard Q-learning: $\phi \leftarrow \phi + \alpha \nabla_{\phi} Q_{\phi}(\mathbf{s}, \mathbf{a})(r(\mathbf{s}, \mathbf{a}) + \gamma V(\mathbf{s}') - Q_{\phi}(\mathbf{s}, \mathbf{a}))$

target value: $V(\mathbf{s}') = \max_{\mathbf{a}'} Q_{\phi}(\mathbf{s}', \mathbf{a}')$

soft Q-learning: $\phi \leftarrow \phi + \alpha \nabla_{\phi} Q_{\phi}(\mathbf{s}, \mathbf{a})(r(\mathbf{s}, \mathbf{a}) + \gamma V(\mathbf{s}') - Q_{\phi}(\mathbf{s}, \mathbf{a}))$

target value: $V(\mathbf{s}') = \text{soft max}_{\mathbf{a}'} Q_{\phi}(\mathbf{s}', \mathbf{a}') = \log \int \exp(Q_{\phi}(\mathbf{s}', \mathbf{a}')) d\mathbf{a}'$

$\pi(\mathbf{a}|\mathbf{s}) = \exp(Q_{\phi}(\mathbf{s}, \mathbf{a}) - V(\mathbf{s})) = \exp(A(\mathbf{s}, \mathbf{a}))$

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i')$, add it to $\mathcal{R}$
 2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}_j'\}$ from $\mathcal{R}$ uniformly
 3. compute $y_j = r(\mathbf{s}_j, \mathbf{a}_j) + \gamma \text{soft max}_{\mathbf{a}_j'} Q_{\bar{\theta}}(\mathbf{s}_j', \mathbf{a}_j')$ using *target* network $Q_{\bar{\theta}}$
 4. $\theta \leftarrow \theta - \alpha \sum_j \frac{dQ_{\theta}}{d\theta}(\mathbf{s}_j, \mathbf{a}_j)(Q_{\theta}(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
 5. update $\bar{\theta}$: copy θ every N steps

Policy gradient with soft optimality

Recall:

$$\log p(\mathcal{O}_{1:T}) \geq \sum_t E_{(\mathbf{s}_t, \mathbf{a}_t) \sim q} [r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(q(\mathbf{a}_t | \mathbf{s}_t))] = \mathcal{L}(q)$$

Policy gradient objective:

$$J(\theta) = \sum_t E_{(\mathbf{s}_t, \mathbf{a}_t) \sim \pi_\theta} [r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(\pi_\theta(\mathbf{a}_t | \mathbf{s}_t))]$$

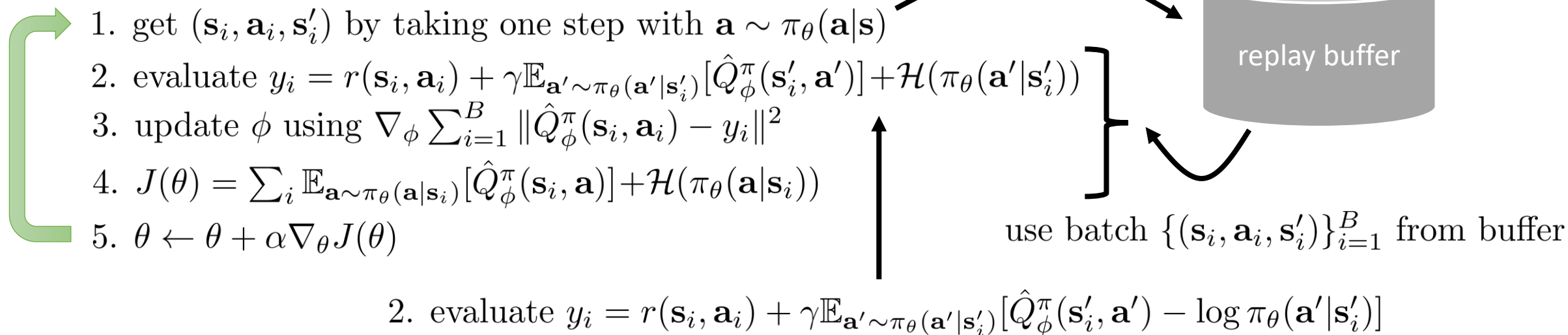
it is very common to add an entropy term to policy gradient

entropy has a closed form expression for most common policy classes

Soft actor-critic

What if we want an *actor-critic* algorithm with soft optimality?

off-policy actor-critic algorithm:



This is one of the most commonly used off-policy actor-critic algorithms!

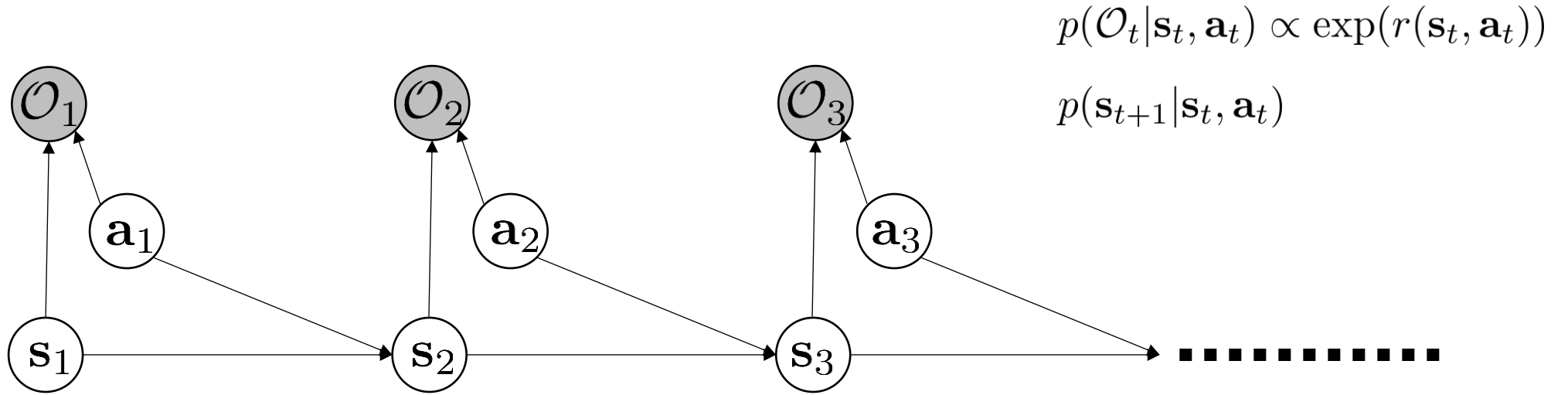
usually put a “temperature” β in front of $\mathcal{H}(\pi_\theta(\mathbf{a}|\mathbf{s}_i))$

why?

Part 4:

Inverse reinforcement learning

Inference = planning



1. compute backward messages $\beta_t(s_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | s_t, \mathbf{a}_t)$
2. compute policy $p(\mathbf{a}_t | s_t, \mathcal{O}_{1:T})$

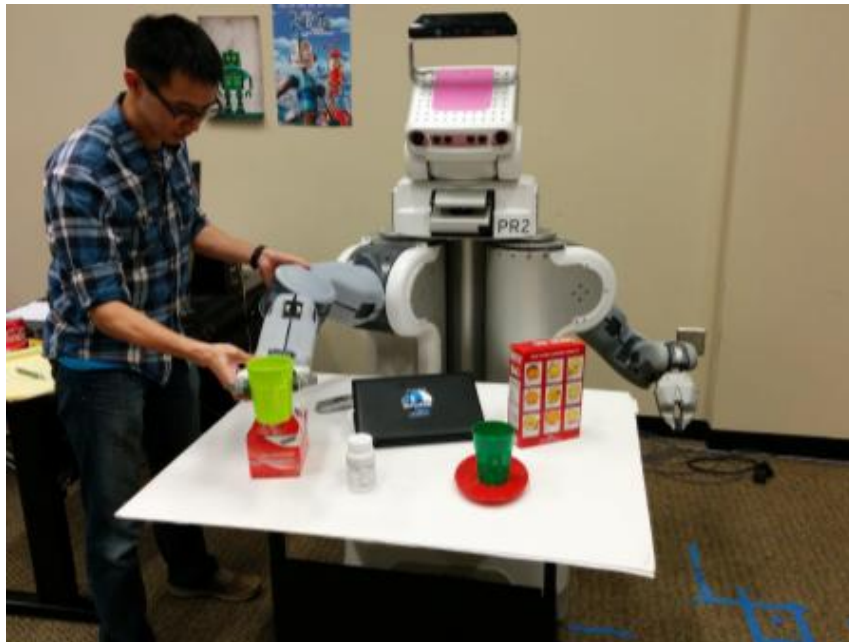
can we invert this, and learn a reward function from demonstrations *assuming* that the demonstrator uses this model?

Why should we worry about learning rewards?

The imitation learning perspective

Standard imitation learning:

- copy the *actions* performed by the expert
- no reasoning about outcomes of actions



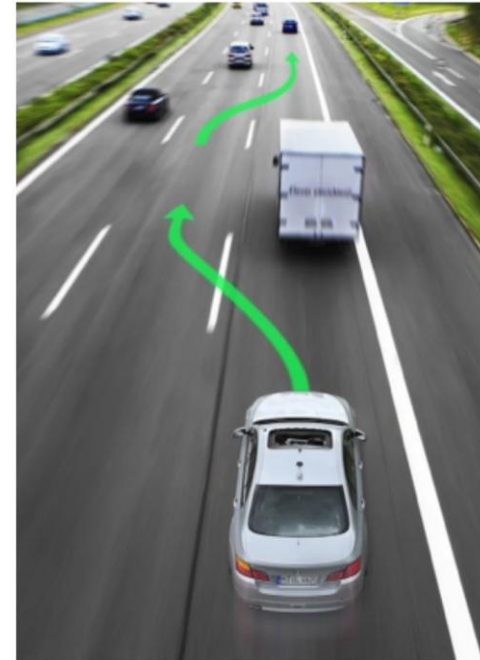
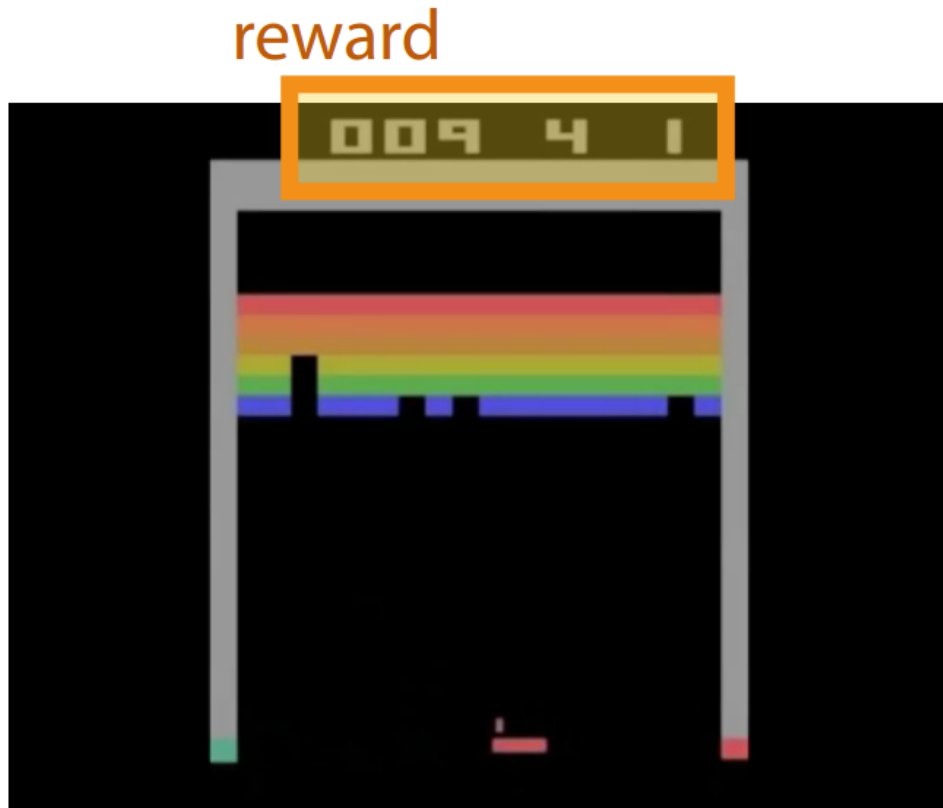
Human imitation learning:

- copy the *intent* of the expert
- might take very different actions!



Why should we worry about learning rewards?

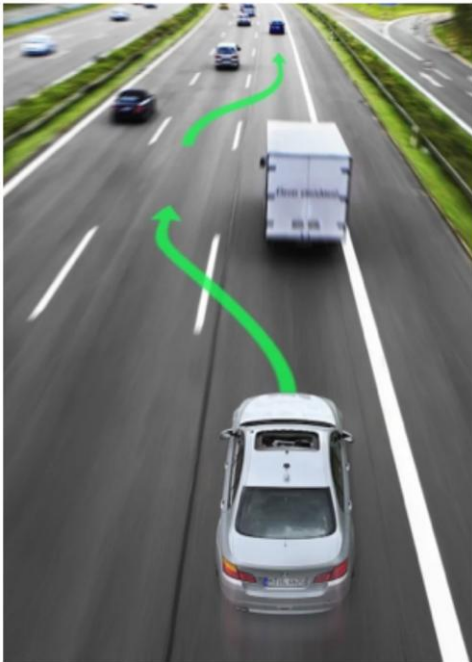
The reinforcement learning perspective



what is the reward?

Inverse reinforcement learning

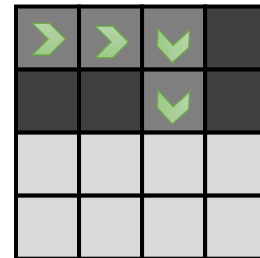
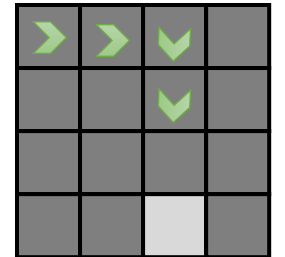
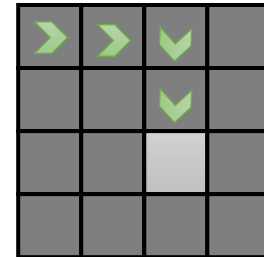
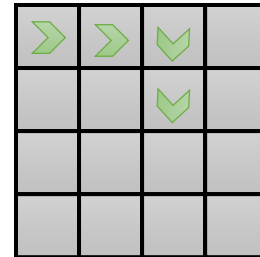
Infer **reward functions** from **demonstrations**



$r(\mathbf{s}, \mathbf{a})$

by itself, this is an **underspecified** problem

many reward functions can explain the **same** behavior



Learning the optimality variable

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t, \psi) \propto \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t))$$

$\nearrow$
reward parameters

given:

samples $\{\tau_i\}$ sampled from $\pi^*(\tau)$

$$p(\tau | \mathcal{O}_{1:T}, \psi) \propto \cancel{p(\tau)} \exp\left(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\right)$$

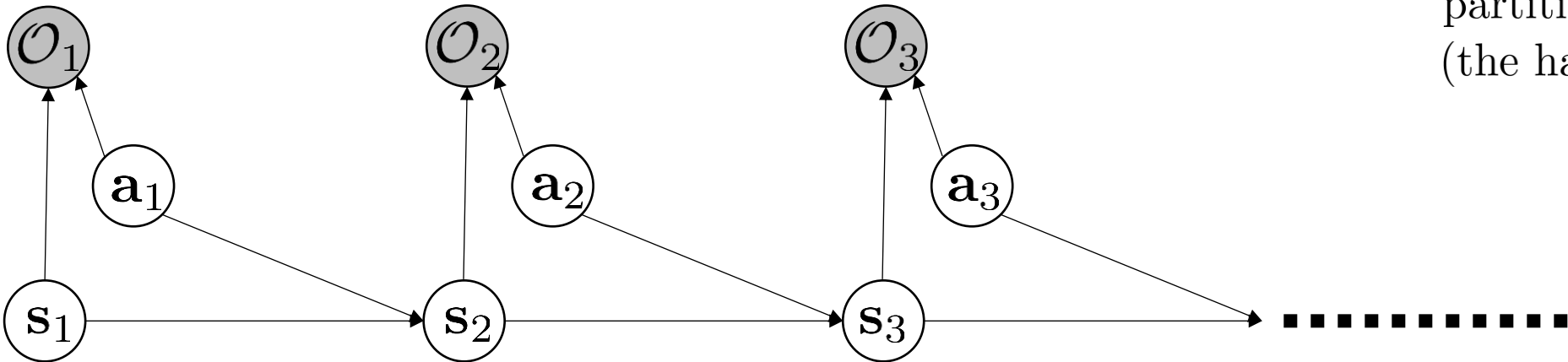
can ignore (independent of ψ)

$\searrow$

maximum likelihood learning:

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N \log p(\tau_i | \mathcal{O}_{1:T}, \psi) = \max_{\psi} \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \log Z$$

$\nwarrow$
partition function (the hard part)



The IRL partition function

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N r_{\psi}(\tau_i) - \log Z$$

$$Z = \int p(\tau) \exp(r_{\psi}(\tau)) d\tau$$

$$\nabla_{\psi} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \underbrace{\frac{1}{Z} \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau}_{p(\tau | \mathcal{O}_{1:T}, \psi)}$$

$$\nabla_{\psi} \mathcal{L} = E_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)] - E_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]$$



estimate with expert samples



soft optimal policy under current reward

Estimating the gradient

$$\nabla_{\psi} \mathcal{L} = E_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)] - E_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]$$



estimate with expert samples



soft optimal policy under current reward

idea: learn $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$ using any max-ent RL algorithm

$$J(\theta) = \sum_t E_{\pi(\mathbf{s}_t, \mathbf{a}_t)} [r_{\psi}(\mathbf{s}_t, \mathbf{a}_t)] + E_{\pi(\mathbf{s}_t)} [\mathcal{H}(\pi(\mathbf{a} | \mathbf{s}_t))]$$

then run this policy to sample $\{\tau_j\}$

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{M} \sum_{j=1}^M \nabla_{\psi} r_{\psi}(\tau_j)$$



sum over expert samples



sum over policy samples

More efficient sample-based updates

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{M} \sum_{j=1}^M \nabla_{\psi} r_{\psi}(\tau_j)$$


sum over expert samples

sum over policy samples

improve ~~learn~~ $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$ using any max-ent RL algorithm
(a little)
then run this policy to sample $\{\tau_j\}$

looks expensive! what if we use “lazy” policy optimization?

problem: estimator is now biased! wrong distribution!

solution 1: use importance sampling

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_{\psi} r_{\psi}(\tau_j) \quad w_j = \frac{p(\tau) \exp(r_{\psi}(\tau_j))}{\pi(\tau_j)}$$

Importance sampling

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_{\psi} r_{\psi}(\tau_j)$$

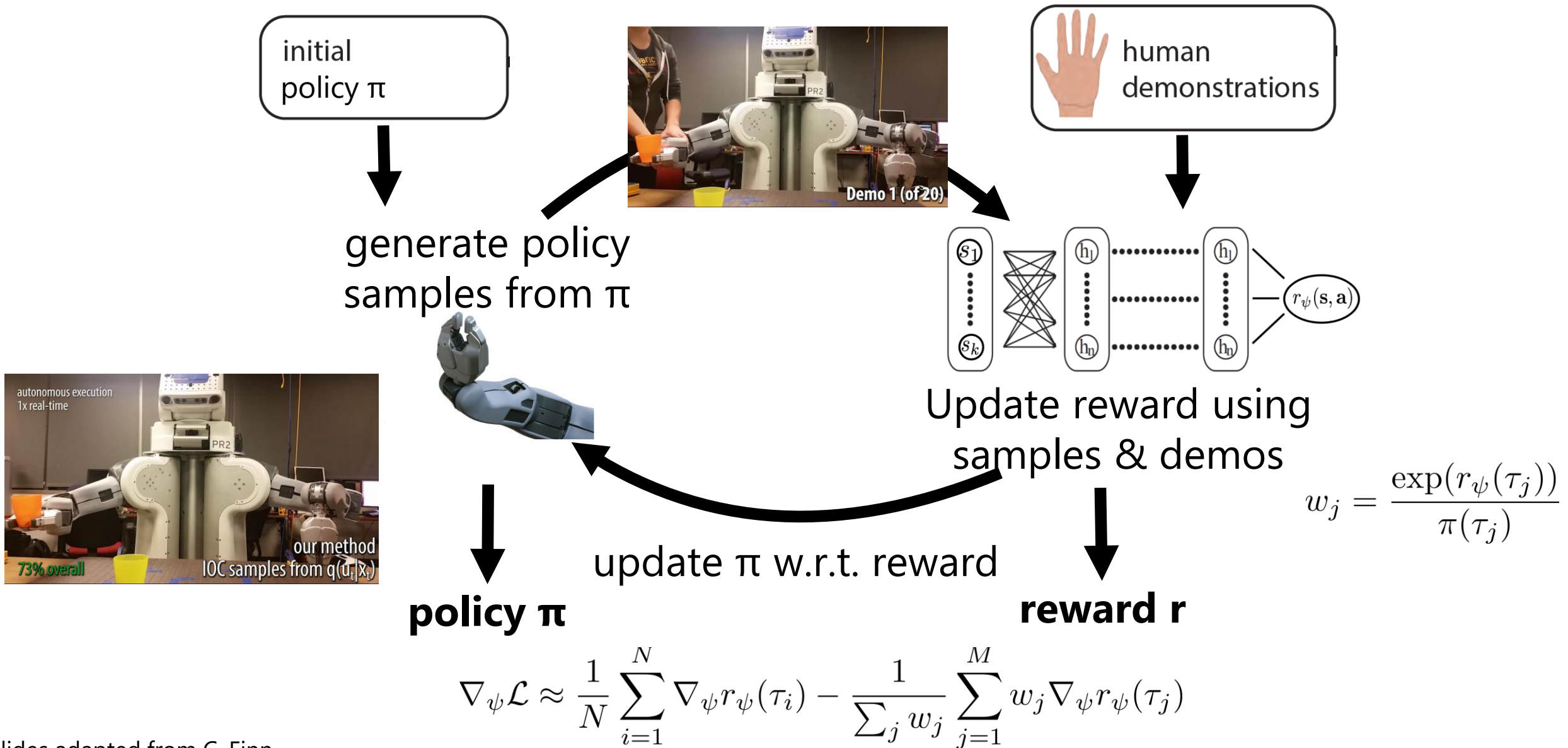
$$w_j = \frac{p(\tau) \exp(r_{\psi}(\tau_j))}{\pi(\tau_j)}$$

$$\begin{aligned} & \frac{\cancel{p(\mathbf{s}_1)} \prod_t \cancel{p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)} \exp(r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{\cancel{p(\mathbf{s}_1)} \prod_t \cancel{p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)} \pi(\mathbf{a}_t|\mathbf{s}_t)} \\ &= \frac{\exp(\sum_t r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{\prod_t \pi(\mathbf{a}_t|\mathbf{s}_t)} \end{aligned}$$

each policy update w.r.t. r_{ψ} brings us closer to the target distribution!

guided cost learning algorithm

(Finn et al. ICML '16)



Part 2:

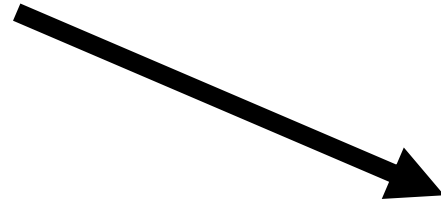
Adversarial imitation learning

It looks a bit like a game...

initial
policy π



samples from $\pi_\theta(\tau)$



$$\nabla_\theta \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^M \nabla_\theta \log \pi_\theta(\tau_j) r_\psi(\tau_j)$$

policy changed to make it *harder* to distinguish from demos

 human
demonstrations



samples from $\pi^*(\tau)$



$$\nabla_\psi \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_\psi r_\psi(\tau_j)$$

demos are made more likely, samples less likely

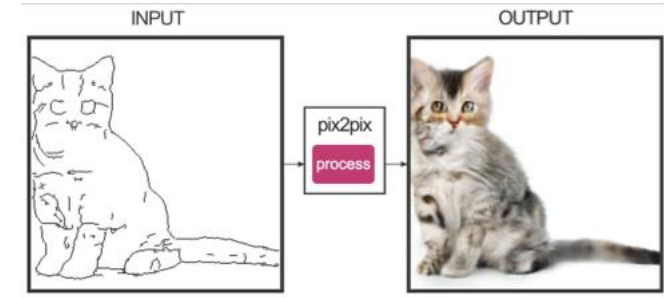
Generative Adversarial Networks



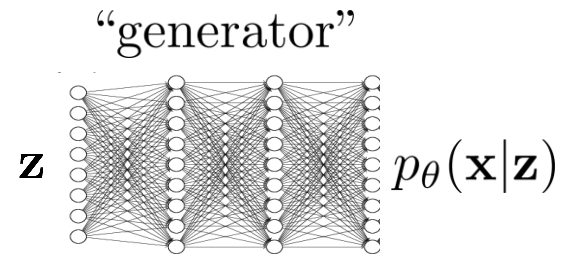
Zhu et al. '17



Arjovsky et al. '17



Isola et al. '17



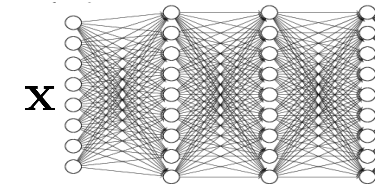
samples from $p_\theta(\mathbf{x})$

data (“demonstrations”)



samples from $p^*(\mathbf{x})$

$$D(\mathbf{x}) = p_\psi(\text{real image}|\mathbf{x})$$

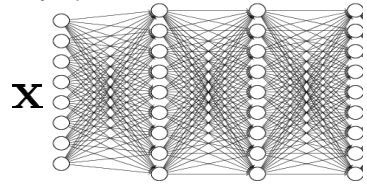


$$\psi = \arg \max_{\psi} \frac{1}{N} \sum_{\mathbf{x} \sim p^*} \log D_\psi(\mathbf{x}) + \frac{1}{M} \sum_{\mathbf{x} \sim p_\theta} \log(1 - D_\psi(\mathbf{x}))$$

$$\theta \leftarrow \arg \max_{\theta} E_{\mathbf{x} \sim p_\theta} \log D_\psi(\mathbf{x})$$

Inverse RL as a GAN

$$D(\mathbf{x}) = p_\psi(\text{real image}|\mathbf{x})$$



which discriminator is best?

$$D^*(\mathbf{x}) = \frac{p^*(\mathbf{x})}{p_\theta(\mathbf{x}) + p^*(\mathbf{x})}$$

for IRL, optimal policy approaches $\pi_\theta(\tau) \propto p(\tau) \exp(r_\psi(\tau))$

choose this parameterization for discriminator:

optimize Z w.r.t. same objective as ψ !

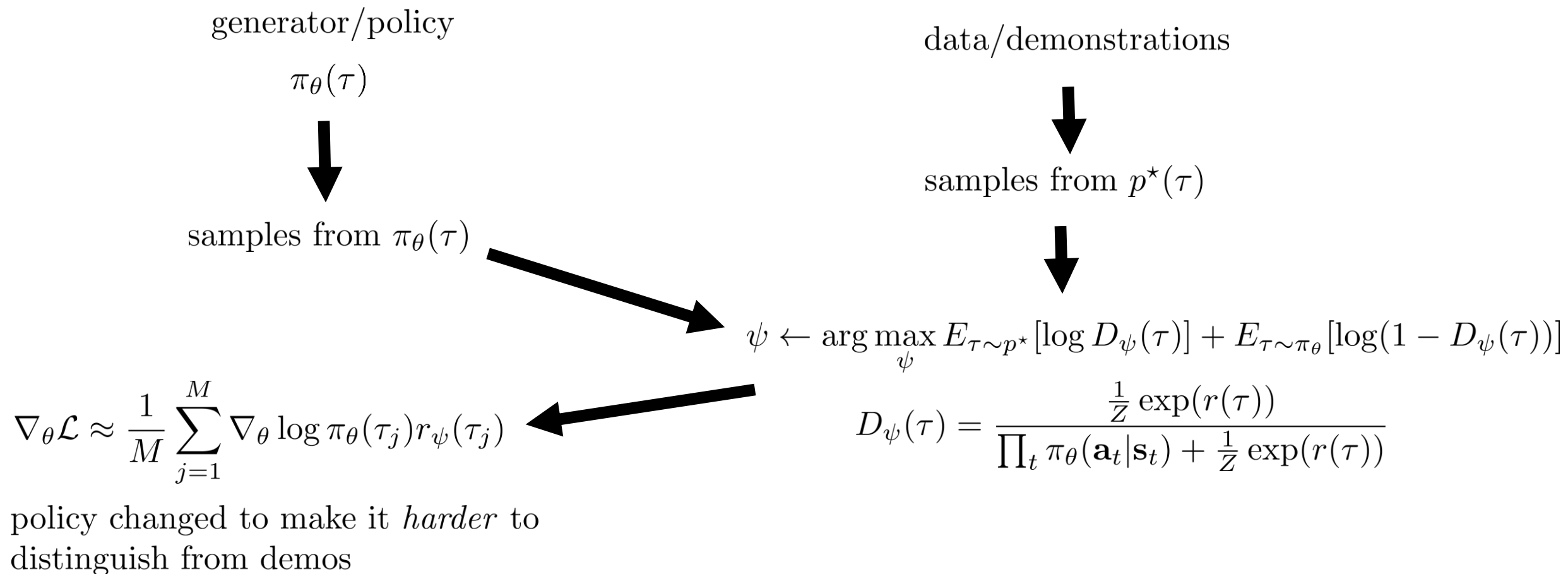
$$D_\psi(\tau) = \frac{p(\tau) \frac{1}{Z} \exp(r(\tau))}{p_\theta(\tau) + p(\tau) \frac{1}{Z} \exp(r(\tau))} = \frac{\cancel{p(\tau)} \frac{1}{Z} \exp(r(\tau))}{\cancel{p(\tau)} \prod_t \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) + \cancel{p(\tau)} \frac{1}{Z} \exp(r(\tau))} = \frac{\frac{1}{Z} \exp(r(\tau))}{\prod_t \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) + \frac{1}{Z} \exp(r(\tau))}$$

optimize this w.r.t. ψ

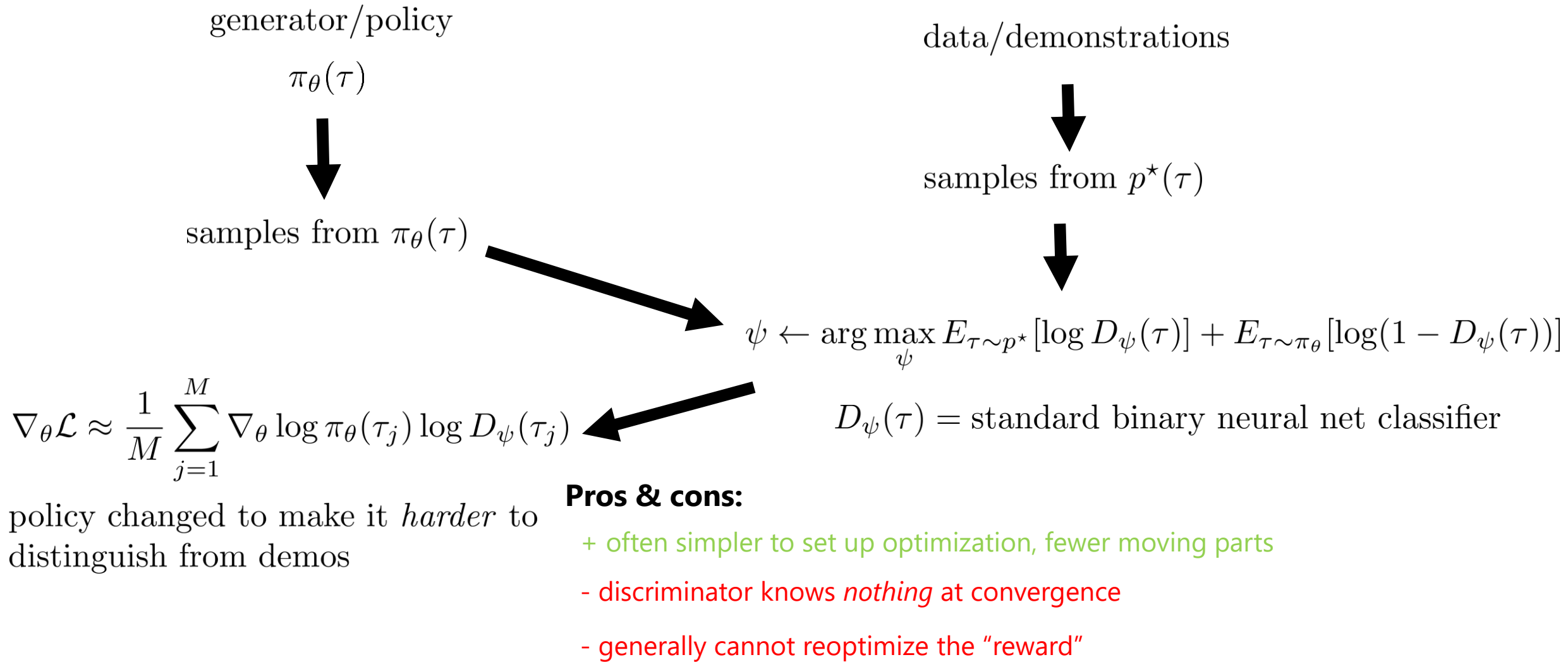
$$\psi \leftarrow \arg \max_{\psi} E_{\tau \sim p^*} [\log D_\psi(\tau)] + E_{\tau \sim \pi_\theta} [\log(1 - D_\psi(\tau))]$$

we don't need importance weights anymore – they are subsumed into Z

Inverse RL as a GAN



Can we just use a regular discriminator?



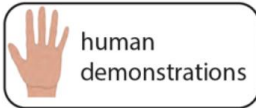
IRL as adversarial optimization

minimized maximized

reward function



robot attempt



human demonstrations

learns distribution $p(\tau)$ such that
demos have max likelihood
 $p(\tau) \propto \exp(r(\tau))$ (MaxEnt model)

$$D(\tau) = \frac{\frac{1}{Z} \exp(r(\tau))}{\frac{1}{Z} \exp(r(\tau)) + \pi(\tau)}$$

actually the
same thing!

False True

classifier



robot attempt

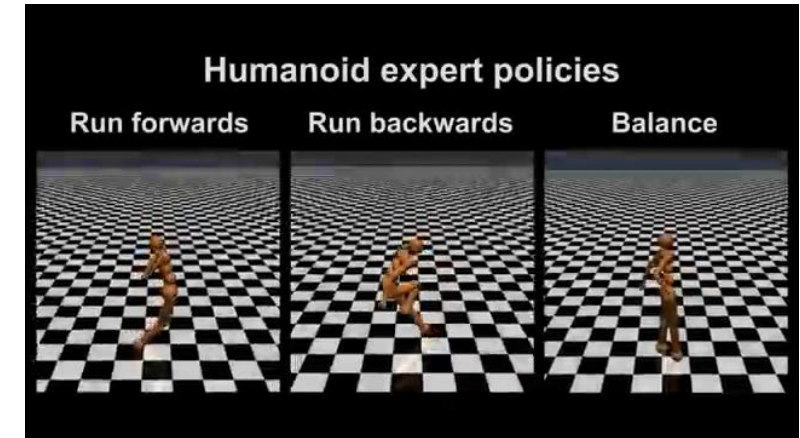


human demonstrations

$D(\tau)$ = probability τ is a demo

use $\log D(\tau)$ as “reward”

$D(\tau)$ = some classifier



Hausman, Chebotar, Schaal, Sukhatme, Lim

Motion Imitation



Reference Motion
(Mocap)



Learned Policy
(Simulation)

the goal is to train a simulated character to imitate the motion.

Peng, Kanazawa, Toyer, Abbeel, Levine